

Relazione di Progetto di Sistemi Operativi

Applicazione Elenco Telefonico con Socket TCP

Giacomo Abbruciati - Matricola: 0356240

Indice

1	Introduzione, Specifiche e Scelte progettuali	2
1.1	Specifiche del progetto	2
1.2	Scelte progettuali	2
1.2.1	Comunicazione Client $\longleftrightarrow$ Server	2
1.2.2	Livelli di Autorizzazione	2
1.2.3	Strutture dati e Ricerca di Contatti	2
2	Specifica del Protocollo	3
2.1	Struttura Header di Pacchetto	3
2.2	Opcode di richiesta	3
2.2.1	Opcode LOGIN & REGISTER	4
2.2.2	Opcode INSERT	4
2.2.3	Opcode DELETE	4
2.2.4	Opcode SEARCH	5
2.3	Status code di risposta	5
3	Specifica del Server	6
3.1	Architettura Concorrente	6
3.2	Autenticazione e persistenza	6
3.3	Base di dati	6
3.4	Sincronizzazione	6
3.5	Elevazione dei permessi	7
4	Specifica del Client	7
5	Compilazione e Avvio	7

1 Introduzione, Specifiche e Scelte progettuali

1.1 Specifiche del progetto

La specifica del progetto richiede che venga sviluppato un servizio di "Elenco Telefonico", supportato da un server che gestisca le richieste provenienti da client residenti su macchine diverse.

L'applicazione sviluppata dovrà supportare la ricerca e l'aggiunta di record all'interno dell'elenco telefonico se si è sufficientemente autorizzati.

La specifica richiede inoltre lo sviluppo di un'applicazione client in grado di interagire con il server per effettuare le operazioni richieste.

1.2 Scelte progettuali

1.2.1 Comunicazione Client $\longleftrightarrow$ Server

Per l'interazione tra il client ed il server si è scelto di sviluppare un protocollo applicativo ad-hoc la cui specifica è descritta nella sezione successiva.

Il client ed il server scambiano messaggi tramite *Socket TCP*: questo permette alle due applicazioni di risiedere su macchine diverse collocate in aree geografiche diverse.

1.2.2 Livelli di Autorizzazione

Poiché nella specifica viene richiesto che gli utenti che sono autorizzati ad inserire contatti non siano generalmente gli stessi autorizzati a ricercarli, si è optato per una struttura che prevede dei livelli di autorizzazione per la gestione delle operazioni effettuabili.

Ruolo	Ricerca contatti	Inserimento contatti	Eliminazione contatti
Anonymous	✗	✗	✗
User	✓	✗	✗
Admin	✓	✓	✓

Tabella 1: Descrizione dei Permessi Utente

Quando un utente si registra, a questo viene assegnato automaticamente il ruolo *Anonymous*: questo non permette all'utente di svolgere alcuna operazione.

Il server tramite la sua console ha la possibilità di modificare i permessi di un utente, cambiando i privilegi.

Come descritto nella Tabella 1 un utente di tipo *User* ha l'unica possibilità di ricercare contatti nell'elenco telefonico mentre per l'inserimento è necessario un livello di autorizzazione *Admin*.

1.2.3 Strutture dati e Ricerca di Contatti

Per individuare gli utenti che possono accedere al sistema si è scelto *Username* e *Password*. Lo *Username* ha un limite massimo di 32 caratteri.

Per quanto riguarda la gestione della *Password* questa dopo essere inserita dall'utente viene convertita nel suo digest SHA-256 per garantire una dimensione fissa di 32 byte.

Per implementare la specifica richiesta si è scelto di assegnare ad ogni contatto presente nell'Elenco Telefonico nome e numero di telefono, questi sono entrambi rappresentati come stringhe limitate nella loro lunghezza rispettivamente a 128 e 20 caratteri.

Per effettuare una ricerca all'interno del sistema il client invia al server una richiesta di ricerca contenente il numero massimo di contatti che si aspetta vengano ritornati ed il nome del contatto che si desidera trovare. Il server eseguirà una ricerca all'interno del database ritornando al richiedente tutti i contatti trovati utilizzando una ricerca per prefisso di tipo *case-insensitive* sul campo nome.

Poiché la dimensione del campo dell'header relativo alla lunghezza del *Payload* è fissata a 2 byte, la quantità di dati relativi ai contatti restituiti dal server verrà troncata affinché non superi il limite massimo di $2^{16} - 1 = 65\,535$ byte.

2 Specifica del Protocollo

Il protocollo che si è deciso di sviluppare si appoggia sul protocollo di trasporto *TCP* per la garanzia di consegna dei pacchetti.

Si è optato per uno schema *Header – Payload* con un *Header* fisso a 32 bit e un *Payload* variabile a seconda del tipo di messaggio. L'*Header* contiene un marcatore d'inizio per scartare immediatamente le richieste malformate; questo contiene anche il tipo di messaggio inviato e la lunghezza di un eventuale *Payload*.

2.1 Struttura Header di Pacchetto

L'header di pacchetto contiene le informazioni relative alla natura del messaggio e alla lunghezza del relativo *Payload*.

Il primo byte rappresenta una sequenza di riconoscimento del pacchetto, per verificare che la provenienza della richiesta sia stata originata da un client adeguato, filtrando così messaggi errati causati ad esempio dalla connessione tramite *HTTP* al server. Questo byte è stato fissato a 0xA5.

Il secondo byte rappresenta l'*Opcode* (0x00–0x04) della richiesta se il messaggio è in ingresso al server, oppure lo *Status Code* (0x80–0x89) se il messaggio è in uscita dal server.

I successivi due byte indicano la lunghezza del *Payload*, ovvero il contenuto effettivo del messaggio.

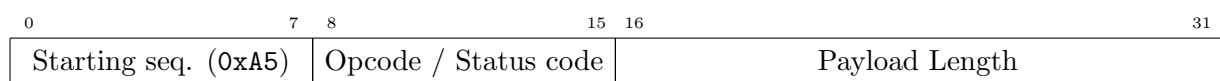


Figura 1: Struttura dell'Header di Pacchetto

2.2 Opcode di richiesta

Una richiesta ha come primo byte informativo l'*Opcode*: questo definisce il tipo di operazione che si intende eseguire, nella Tabella 2 sono mostrate le operazioni disponibili.

Hex	Opcode	Parametri	Descrizione
0x00	LOGIN	<username> <password_hash>	Effettua l'accesso
0x01	REGISTER	<username> <password_hash>	Registra un utente
0x02	INSERT	<name> <phone_number>	Inserisce un contatto
0x03	DELETE	<name>	Rimuove un contatto
0x04	SEARCH	<limit> <query>	Ricerca un contatto

Tabella 2: Lista di opcode disponibili

2.2.1 Opcode LOGIN & REGISTER

Gli *Opcode* LOGIN e REGISTER individuano rispettivamente le operazioni di Accesso e Registrazione al servizio di elenco telefonico. Il *Payload* è strutturato ponendo nei primi Payload Length–32 byte l’username dell’utente e negli ultimi 32 byte l’hash della password effettuato dal client.

Username	Password (32 Byte)
----------	--------------------

Figura 2: Struttura del *Payload* di una richiesta LOGIN o REGISTER

A seguito di una richiesta di tipo LOGIN, se l’autenticazione avrà successo, il client aspetterà una risposta per cui il primo e unico byte del *Payload* conterrà il livello di autorizzazione dell’utente che ha effettuato l’accesso. Nel caso l’autenticazione non abbia successo lo *Status Code* sarà INVALID_CREDENTIALS il *Payload* della risposta avrà dimensione nulla.

Per una richiesta di tipo REGISTER ciò non avviene poiché un utente appena registrato si presuppone avere il livello di autorizzazione minimo, per cui in questo caso il *Payload* avrà dimensione nulla.

0	7	8	15	16	31	32	40
Starting seq. (0xA5)				Status Code (OK)			
				Payload Length (0x0001)			
				Auth Level			

(a) Struttura del *Payload* di risposta di una richiesta LOGIN con esito positivo.

0	7	8	15	16	31
Starting seq. (0xA5)				Status Code	
				Payload Length (0x0000)	

(b) Struttura del *Payload* di risposta di una richiesta REGISTER o di una richiesta LOGIN senza successo.

2.2.2 Opcode INSERT

L’*Opcode* INSERT individua l’operazione di inserimento di un nuovo contatto nell’Elenco telefonico. Il primo byte del *Payload* di questo tipo di pacchetto individua la lunghezza in byte del nome del contatto da inserire, i seguenti *N* byte saranno quindi costituiti dal nome di questo mentre i restanti byte saranno attribuiti al numero di telefono.

0	7	8
Name Length	Name	Phone Number

Figura 4: Struttura del *Payload* di una richiesta INSERT.

Una richiesta di tipo INSERT che ha successo sarà seguita da una risposta con *Status code* OK. Se l’utente non dovesse essere autorizzato ad eseguire questa operazione la risposta avrà uno *Status Code* UNAUTHORIZED. In entrambi i casi il *Payload* sarà vuoto.

2.2.3 Opcode DELETE

L’*Opcode* DELETE individua l’operazione di rimozione di un contatto dall’Elenco telefonico. Il *Payload* di questo tipo di pacchetto contiene unicamente il *Name* del contatto che si intende eliminare: questo deve coincidere esattamente con il *Name* del contatto target.

Una richiesta di tipo DELETE che ha successo è seguita da una risposta con *Status code* OK. Se l’utente non dovesse essere autorizzato ad eseguire questa operazione la risposta avrà uno *Status Code* UNAUTHORIZED. Se invece il contatto non dovesse esistere ci si aspetterà una risposta il cui *Status Code* sarà NOT_FOUND. In tutti i casi il *Payload* sarà vuoto.

2.2.4 Opcode SEARCH

L'*Opcode* SEARCH individua l'operazione di ricerca di un contatto all'interno dell'Elenco Telefonico. Il primo byte del *Payload* di una richiesta di questo tipo contiene il limite massimo di contatti che il client desidera siano ritornati, i restanti byte rappresentano il nome del contatto che si desidera trovare.

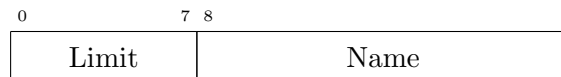


Figura 5: Struttura del *Payload* di una richiesta SEARCH.

Il *Payload* di una risposta di tipo SEARCH è costituito da contatti serializzati sequenzialmente.

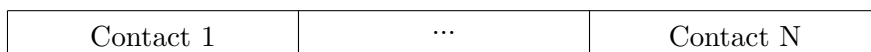


Figura 6: Struttura del *Payload* una risposta SEARCH.

Ciascun contatto è caratterizzato da un *Header* composto da due byte che rappresenteranno rispettivamente la lunghezza del nome e la lunghezza del numero di telefono; a questi due byte seguiranno poi il nome ed il recapito telefonico.

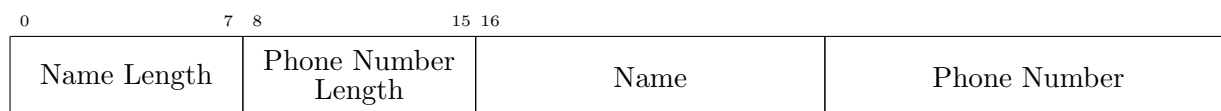


Figura 7: Struttura di un contatto nel *Payload* di una risposta SEARCH.

Questa operazione è accessibile agli utenti con ruolo *User* o *Admin*, un utente con il ruolo *Anonymous* che tenterà di effettuare questa operazione riceverà una risposta con status code UNAUTHORIZED.

2.3 Status code di risposta

Hex	Status	Descrizione	Payload
0x80	OK	Operazione completata con successo	<i>Auth level</i> (1 byte) / —
0x81	SERVER_ERROR	Errore interno al server	—
0x82	MALFORMED_REQUEST	Richiesta non valida	—
0x83	NOT_FOUND	Risorsa non trovata	—
0x84	UNAUTHORIZED	Utente non autorizzato	—
0x85	INVALID_CREDENTIALS	Credenziali inserite non valide	—
0x86	USERNAME_TAKEN	Username già registrato	—
0x87	ALL_RETURNED	Ricerca completata (risultati totali)	Elenco contatti trovati
0x88	FEWER_RETURNED	Ricerca completata (risultati parziali)	Elenco contatti trovati
0x89	CONTACT_ALREADY_EXISTS	Contatto già presente	—

Tabella 3: Codici di stato restituiti dal Server e relativo Payload.

3 Specifica del Server

3.1 Architettura Concorrente

Il server è stato progettato per gestire concorrentemente più connessioni: il thread principale è in ascolto sulla porta adibita al servizio (5050), una volta stabilita la connessione, viene creato un thread apposito che gestirà la comunicazione con il client.

3.2 Autenticazione e persistenza

L'autenticazione di un utente avviene tramite una richiesta di tipo **LOGIN** al server, l'autenticazione persiste per tutta la connessione fino alla disconnessione del client.

Nel caso in cui il client inoltri un'ulteriore richiesta di **LOGIN** l'autenticazione se di successo cambierà l'identità del client.

3.3 Base di dati

Il salvataggio dei dati (credenziali degli utenti e contatti) avviene rispettivamente su due file su disco.

Si è scelto di organizzare i record sequenzialmente. Per quanto riguarda il database dei contatti, la rimozione di un record espone il database ad un problema di frammentazione esterna; per ovviare a ciò in ogni record è stato inserito un byte, denominato **ACTIVE** ed impostato di default ad 1: questo indica se il record è attivo o è stato eliminato.

Quando si effettua un inserimento si scansiona linearmente il database: se un record eliminato (**ACTIVE** == 0) viene trovato e questo ha la stessa lunghezza del record che si vuole inserire, il nuovo contatto lo sovrascriverà. In caso contrario questo verrà inserito alla fine del file.

0	7 8	15 16	23 24	
Active	Name Length	Phone Number Length	Name	Number

Figura 8: Struttura del record di un contatto.

0	7 8	15 16	47 48
Username Length	Auth Level	Password	Username

Figura 9: Struttura del record di un utente.

3.4 Sincronizzazione

Poiché il server gestisce le richieste in modalità concorrente, ciò espone il database ad un problema di sincronizzazione: questo viene risolto tramite l'ausilio di una logica di gestione semaforica dedicata.

L'obiettivo di questa gestione è di permettere a più lettori di leggere concorrentemente il file di database, impedendo agli scrittori di scrivere se ci sono lettori attivi; poiché questa implementazione potrebbe provocare la starvation degli scrittori, il meccanismo implementato prevede una priorità degli scrittori sui lettori, bloccando questi ultimi fino a che le scritture precedenti non siano state completate.

L'implementazione prevede quindi l'utilizzo di tre semafori **SYSTEM V** per permettere l'atomicità tra più operazioni semaforiche:

- `SEM_READERS`: semaforo per i lettori attivi, inizializzato a 0
- `SEM_WRITERS_WAITING`: semaforo per gli scrittori in attesa di scrittura, inizializzato a 0
- `MUTEX_WRITING`: mutex per permettere solo ad uno scrittore di scrivere, inizializzato ad 1

Il lettore si muoverà secondo questo schema:

1. Atomicamente attendi `SEM_WRITERS_WAITING == 0` e incrementa `SEM_READERS += 1`
2. Leggi
3. Decrementa `SEM_READERS -= 1`

Lo scrittore si muoverà secondo questo schema:

1. Incrementa `SEM_WRITERS_WAITING += 1`
2. Prendi il mutex `MUTEX_WRITING`
3. Attendi `SEM_READERS == 0`
4. Scrivi
5. Decrementa `SEM_WRITERS_WAITING -= 1`
6. Rilascia il mutex `MUTEX_WRITING`

3.5 Elevazione dei permessi

All'avvio del server, questo inizierà un thread adibito alla ricezione da `stdin` di caratteri. Se la stringa `'perm'` viene inserita, il server permetterà la modifica dei permessi di un utente.

4 Specifica del Client

Il client è un'applicazione da riga di comando che permette all'utente la connessione e l'interrogazione del server.

All'avvio viene instaurata una connessione al server tramite *Socket TCP*, mostrando all'utente una serie di menù che permettono l'interazione con il server.

5 Compilazione e Avvio

Il progetto necessita dell'installazione della libreria `libssl-dev` per il supporto delle operazioni di hash. Per installare questa libreria è sufficiente eseguire il comando

```
$ sudo apt install libssl-dev
```

Per la compilazione e avvio delle due applicazioni è sufficiente eseguire

```
# Compilazione del Server
$ make server

# Compilazione del Client
$ make client

# Avvio del Server
$ ./server 5050

# Avvio del Client
```

```
$ ./client 127.0.0.1 5050
```